

CUDA Programming Notes (CMPS 297 GPU)

Combined Notes

May 3, 2026

Contents

1 GPU Architecture	3
2 GPU Memory	4
3 Performance Metrics	5
4 Convolution	7
5 Stencil Computation	8
6 Reduction	9
7 Scan (Kogge-Stone)	10
8 Scan (Brent-Kung)	13
9 Histogram	14
10 Merge	15
11 Sort	16
12 Sparse Matrix (COO, CSR)	19
13 Sparse Matrix (ELL, JDS)	22
14 Graph Algorithms I	24
15 Graph Algorithms II	28
16 Warp Synchronization	29
16.1 intra-warp synchronization	29
16.2 shuffle	29
16.3 voting	29
17 Pinned Memory and Streams	31
17.1 Pinned Memory	31
17.2 Stream	31
18 Dynamic Parallelism	33
19 CUDA Potpourri	36
19.1 Multi-GPU programming	36
19.2 Interconnect	36
19.3 Memory management	36
19.4 Events	36
19.5 Tensor cores	37

19.6 Libraries	37
19.7 Other programming interfaces	37
19.8 Other hardware	37
19.9 Comparison with CPU	37
20 Matrix Multiplication Optimization	38

1 GPU Architecture

- Threads are assigned to SMs at block granularity. (All threads in a block are assigned to the same SM.) Each SM has a limited amount of resources.
- Threads within the same block can collaborate in ways that threads in different blocks cannot:
 - barrier synchronization (e.g., `__syncthreads()`)
 - shared memory
 - others
- Thread scheduling: warp. 32 threads in a warp are scheduled together and executed following a SIMD model.
 - control divergence: branches, loops
 - latency hiding: when some threads are stalled, the SM can switch to another warp that is ready to execute.
- Occupancy constraints

GPU Name	K40	M40	P100	V100	A100?	RTX 3090?	RTX 4090?	H100?	RTX 5090?
Microarchitecture	Kepler	Maxwell	Pascal	Volta	Ampere	Ampere	Ada Lovelace	Hopper	Blackwell
Compute Capability	3.5	5.2	6.0	7.0	8.0	8.6	8.9	9.0	10.0
Number of SMs	15	24	56	80	108	82	128	144	192
Cores per SM	192	128	64	64	64	64	128	128	128
Max threads per SM	2048	2048	2048	2048	2048	2048	2048	2048	1536
Max blocks per SM	16	32	32	32	32	16	32	32	32
Max threads per block	1024	1024	1024	1024	1024	1024	1024	1024	1024
Registers per SM	64K	64K	64K	64K	64K	64K	64K	64K	64K
Shared memory per SM	48KB	96KB	64KB	96KB	164KB	128KB	128KB	228KB	256KB

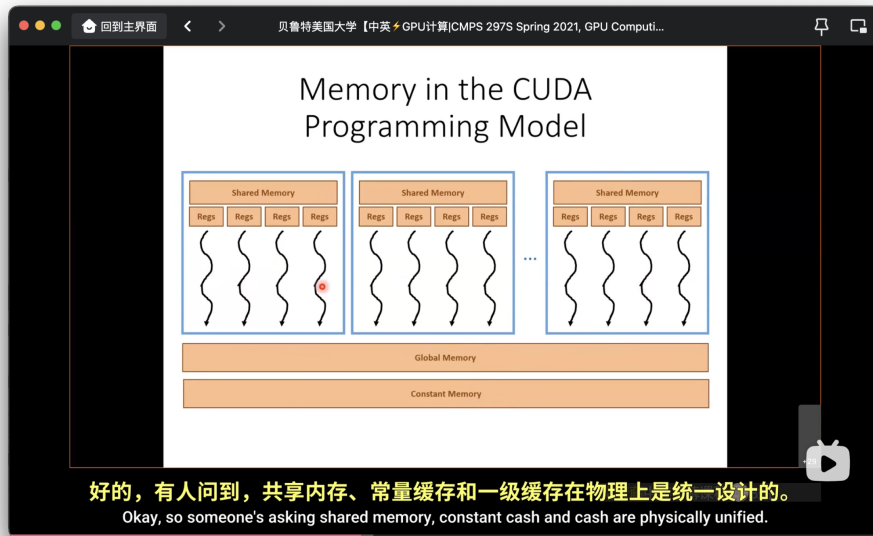


Figure 1: Memory in the CUDA Programming Model

2 GPU Memory

(v100 era)

- performance metrics
 - FLOPS rate; compute-bound
 - memory bandwidth; memory-bound
- Example: matrix multiplication
 - for $N \times N$ matrices
 - * Data loaded: $8N^2$ B
 - * Operations: $2N^3$ OP
 - * potential compute-to-memory ratio: $0.25N$ OP/B
 - opportunity: memory accesses in the kernel code to reuse input data loaded from global memory as much as possible
- memory in GPU
 - global memory: ≈ 500 cycles
 - L1 cache: ≈ 5 cycles, managed by hardware
 - shared memory: ≈ 5 cycles, managed by programmer
 - constant cache: ≈ 5 cycles, managed by programmer
 - L2 cache
 - registers: ≈ 1 cycle

The data gets evicted from the L1 cache before another thread tries to load it, more likely than CPU because of the large number of threads and the small size of the L1 cache.

Dynamic shared memory

- declaration: `extern __shared__ float A.s[];`
- configuration: `kernel <<< numBlocks, numThreadsPerBlock, smemPerBlock >>> (...)`

	Variable declaration	Memory	Scope	Lifetime
<code>__device__ int</code>	<code>globalVar;</code>	global	grid	application
<code>__device__ __constant__ int</code>	<code>constantVar;</code>	constant	grid	application
<code>__device__ __shared__ int</code>	<code>sharedVar;</code>	shared	block	block
	<code>int localVar;</code>	register	thread	thread
	<code>int localArr[N];</code>	global	thread	thread

Table 2: CUDA Type Qualifiers

3 Performance Metrics

some Performance Optimizations

- Tuning resource usage to maximize occupancy
 - Threads per block, shared memory per block, registers per thread [p4]
- Minimizing control divergence to increase SIMD efficiency [p4]
- Shared memory tiling to increase data reuse [p5]
- Memory coalescing
- Thread coarsening

DRAM cell: consists of a capacitance that stores a charge and a three-state device (single-transistor) that allows data to be read/written (capacitor discharged/charged).

DRAM array, DRAM bank.

Row Address $\rightarrow$ [Row Decoder] $\rightarrow$ [DRAM Array] $\rightarrow$ [Sense Amplifier] $\rightarrow$ [Column Latches] $\xrightarrow{\text{burst}}$
[with Column Address $\rightarrow$ Mux] $\rightarrow$ Data

Accessing data in the same burst: no need to access the DRAM array again, just the multiplexer.

Memory Coalescing

- When threads in the same warp access consecutive memory locations in the same burst, the accesses can be combined and served by one burst
 - One DRAM transaction is needed
 - Known as **memory coalescing**
 - Appendix: threads with the same *y* value but consecutive *x* values will go to the same warp
- If threads in the same warp access locations not in the same burst, accesses cannot be combined
 - Multiple transactions are needed
 - Takes longer to service data to the warp
 - Sometimes called **memory divergence**

Latency Hiding with Multiple Banks

- Latency can be hidden by having multiple banks
- Need many threads to simultaneously access memory to keep all banks busy
 - by having high occupancy in SMs

Thread Granularity

- So far, parallelization approaches made threads as fine-grain as possible
 - Assign smallest possible unit of parallelism per thread
 - * e.g., one vector element per thread in vector addition
 - * e.g., one output pixel per thread in RGB to gray and in blur

- * e.g., one output element per thread in matrix multiplication
- Advantages: provide hardware with as many threads as possible to fully utilize resources
 - * transparent scalability
- Disadvantages: common operations are performed redundantly across more threads
 - * Suboptimal if threads are getting serialized by hardware
- Thread coarsening: assign multiple units of parallelism per thread
 - Advantages: reduce the price paid for parallelization
 - * redundant memory
 - * redundant computation
 - * synchronization overhead or divergence
 - Disadvantages
 - * underutilize resources
 - interfere with the transparent scalability
 - * more resources per thread may limit occupancy

Checklist of Common Optimizations

Optimization	Improves	Impact on Cores
Tuning resources to maximize occupancy	Latency hiding	More work to hide pipeline latency
Minimizing control divergence	SIMD efficiency	Fewer idle cores during SIMD execution
Uniform memory access pattern	Memory coalescing	Fewer pipeline stalls waiting for memory
Shared memory tiling	Data reuse	Fewer pipeline stalls waiting for memory
Thread coarsening	Price of parallelization	Less redundant work or synchronization
Privatization (<i>covered later</i>)	Contention on common output	Fewer pipeline stalls waiting for memory

Tensions Between Optimizations

- Maximizing occupancy
 - Maximizing occupancy hides pipeline latency, but too many threads may compete for the cache, evicting each others' data (thrashing the cache)
- Shared memory tiling
 - Using more shared memory enables more data reuse, but may limit occupancy
- Thread coarsening
 - Coarsening reduces redundant work, but requires more resources per thread which may limit occupancy

Need to find the sweet spot that achieves the best compromise

Know Your Bottlenecks

- Optimizations trade one resource for another to alleviate the bottleneck
- Make sure to properly diagnose the bottleneck before applying optimizations

4 Convolution

usage: signal/image/video processing, e.g., Gaussian blur, sharpening, edge detection, etc.
a straightforward implementation: a thread per output element
observation: the mask is small, constant, and accessed by all threads,
store in constant memory

- `__constant__ float mask_c[MASK_DIM][MASK_DIM];`
- `cudaMemcpyToSymbol(mask_c, mask, MASK_DIM * MASK_DIM * sizeof(float));`
- can only allocate up to 64KB
- cache coherency

tiling (M*M mask)

- without tiling, ratio of computation to global loads: $(2M^2 \text{ OP}) / (4M^2 \text{ B}) = 0.5 \text{ OP/B}$
- with tiling, dimensions: input = T , output = $T - M + 1$
 - operations per block: $(T - M + 1)^2 \cdot 2M^2 \text{ OP}$
 - global loads per block: $T^2 \cdot 4 \text{ B}$
 - ratio: $0.5M^2 \cdot (1 - (M - 1)/T)^2 \text{ OP/B}$

5 Stencil Computation

- 5-point, 2D; 7-point, 3D
- grid of points stored as 2D or 3D array
- parallelizing
 - (assuming only inputs in bound)
 - Tiling
 - * limited (max 1024 threads)
 - Thread coarsening
 - * plane by plane
 - * only the current slice is truly shared, save shared memory by putting them in registers:
register tiling

6 Reduction

reduction operation: reduces a set of values to one value, is

- associative
- commutative
- have a well-defined identity value

reduction is a fold
parallel reduction

- reduction tree: Every thread adds two elements in each step
 - $\log(N)$ steps and half the threads drop out every step
- segmented reduction
 - tree reduction on each segment - $\log(N)$ partial sums
 - * using stride as loop variable
 - then
 - * launch a new reduction kernel, or
 - * using atomic operations, or
 - * add them on the CPU
 - problems
 - * SIMD: control divergence
 - * memory access pattern: not coalesced
- optimization
 - coalesced access: stride from BLOCK_DIM to 1
 - using shared memory
 - thread coarsening
 - * price paid for parallelism: are synchronization and control divergence
 - * each thread responsible for more data
- Coarsening Benefits
 - If blocks are all executed in parallel:
 - * $\log(N)$ steps, $\log(N)$ synchronizations
 - If blocks serialized by the hardware by a factor of C:
 - * $C * \log(N)$ steps, $C * \log(N)$ synchronizations
 - If blocks are coarsened by a factor of C:
 - * $2 * (C - 1) + \log(N)$ steps, $\log(N)$ synchronizations

Applying Thread Coarsening

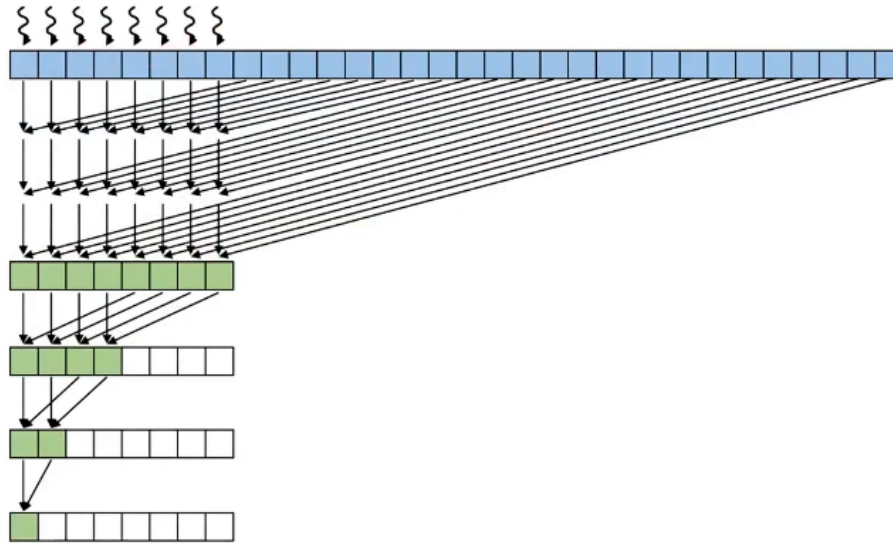


Figure 2: thread coarsening

7 Scan (Kogge-Stone)

scan

scan operation

- takes
 - an input array $[x_0, x_1, \dots, x_{n-1}]$
 - an associative operation $\oplus$, e.g. $+$, $\times$, $\max$, $\min$
- returns
 - an output array $[y_0, y_1, \dots, y_{n-1}]$
 - * inclusive scan: $y_i = x_0 \oplus x_1 \oplus \dots \oplus x_i$
 - * exclusive scan: $y_i = x_0 \oplus x_1 \oplus \dots \oplus x_{i-1}$

segmented scan (also called hierarchical scan)

Kogge-Stone parallel (inclusive) scan

- careful with data races
- `__syncthreads()` in branch: undefined behavior
- shared memory: buffer
- double buffering: don't have to synchronize between read and write

exclusive scan: shift by one, insert identity element at the beginning
work efficiency

- A parallel algorithm is work efficient if it performs the same amount of work as the corresponding sequential algorithm

scan work efficiency

- sequential scan: N additions

Segmented Scan Example

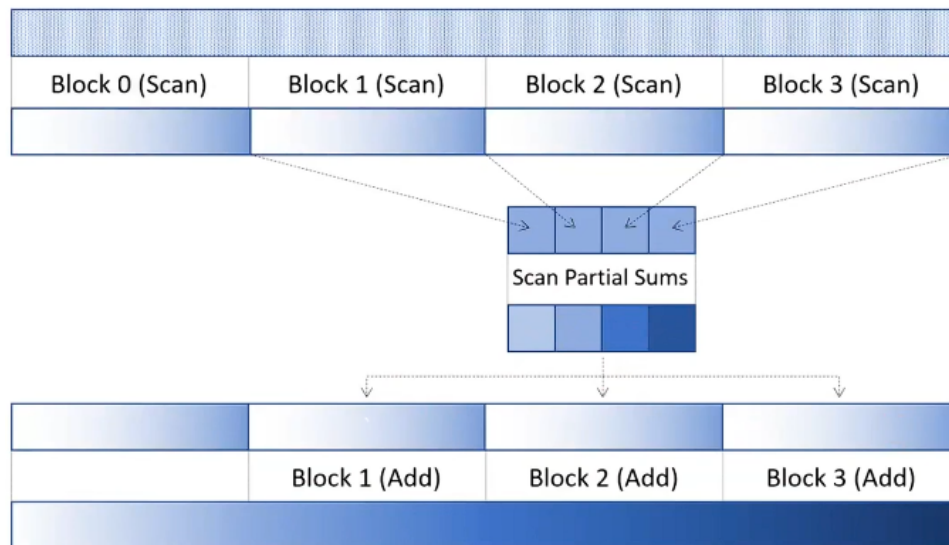


Figure 3: segmented scan

- Kogge-Stone parallel scan
 - $N - 2^{step}$ operations per step, $\log N$ steps
 - total work: $N \log(N) - (N - 1) = \mathcal{O}(N \log N)$ operations
 - not work efficient
- Brent-Kung parallel scan
 - Reduction: $\log(N)$ steps, $N - 1$ operations
 - Post-reduction: $\log(N) - 1$ steps, $N - 2 - (\log(N) - 1)$ operations
 - total: $2\log(N) - 1$ steps, $2N - \log(N) - 2 = \mathcal{O}(N)$ operations
 - takes more steps but more work efficient

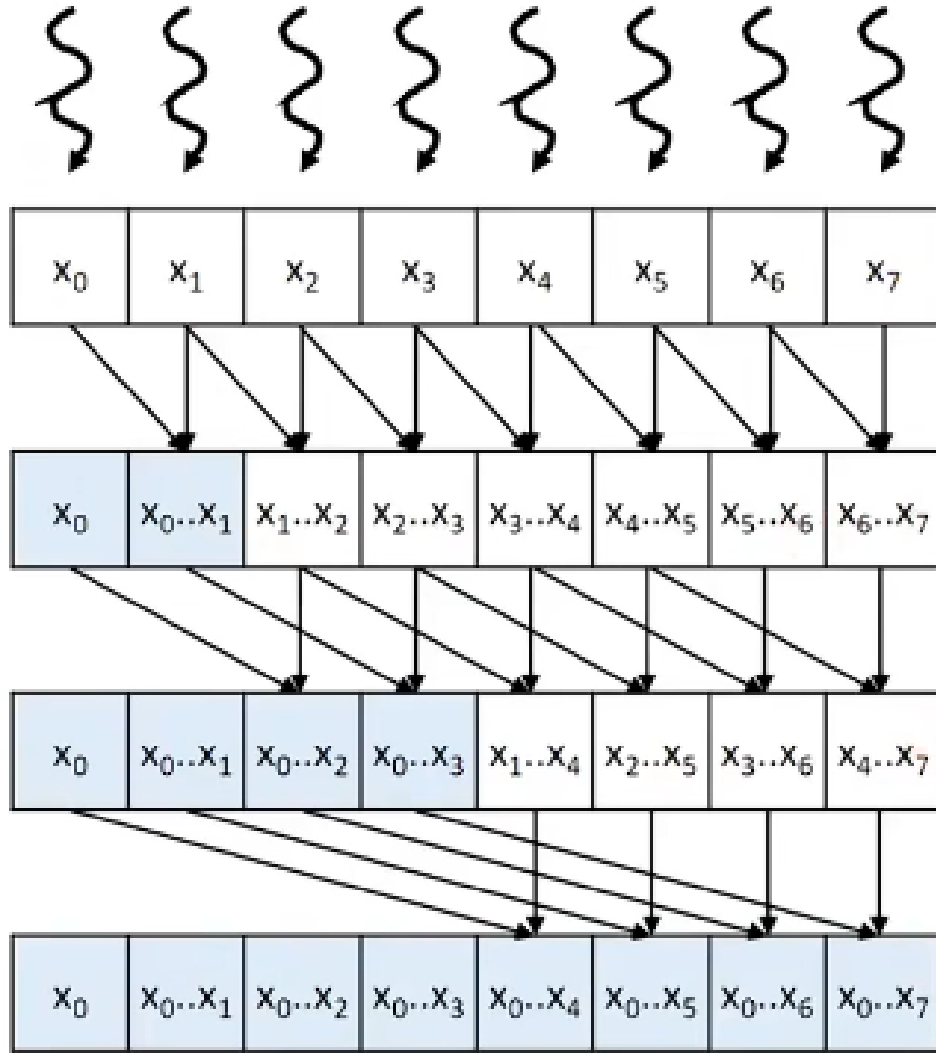


Figure 4: Kogge-Stone parallel (inclusive) scan

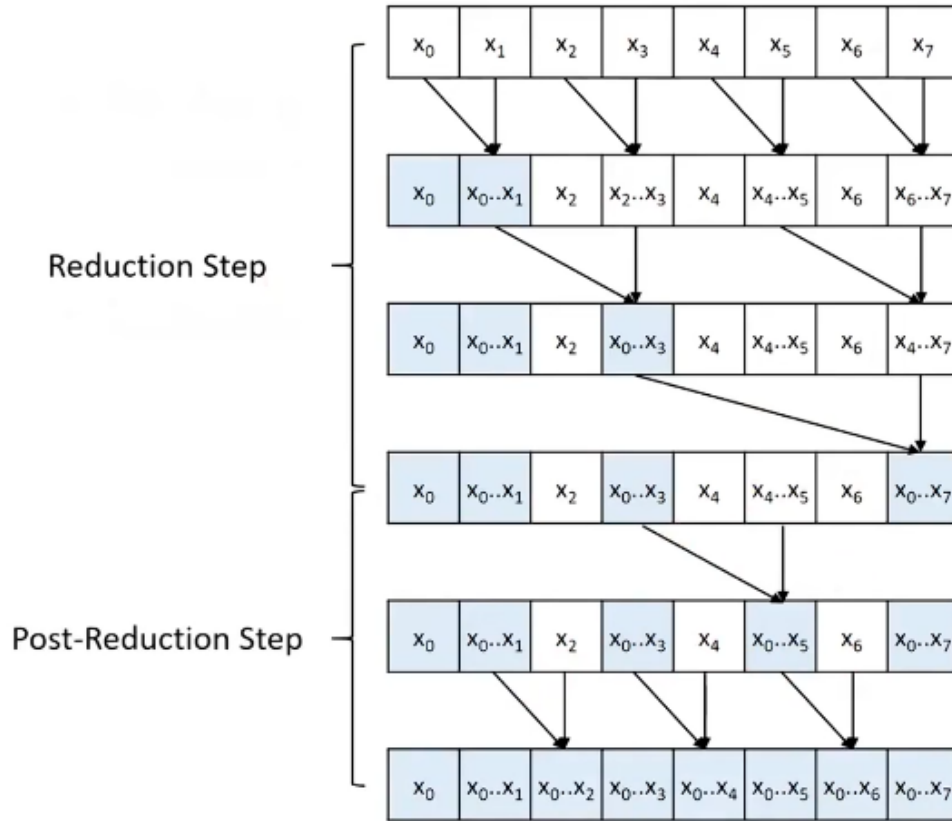


Figure 5: Brent-Kung Parallel (Inclusive) Scan

8 Scan (Brent-Kung)

Brent-Kung Parallel (Inclusive) Scan

- using shared memory also enables coalescing (already coalesced in Kogge-Stone)
- no need double-buffering
- control divergence
 - don't assign threads to specific data elements
 - re-index threads on every iteration to different elements
 - * reduction: using the right operand as the index
 - * post-reduction: using the left operand as the index
 - * when stride is s , thread t is responsible for element $(t + 1) \cdot (2s) - 1$

exclusive scan

- shift elements, or
- different post-reduction step: Brent-Kung Parallel (Exclusive) Scan

Thread Coarsening for scan

- each thread scans a segment sequentially
- only scan the partial sums of each segment in parallel

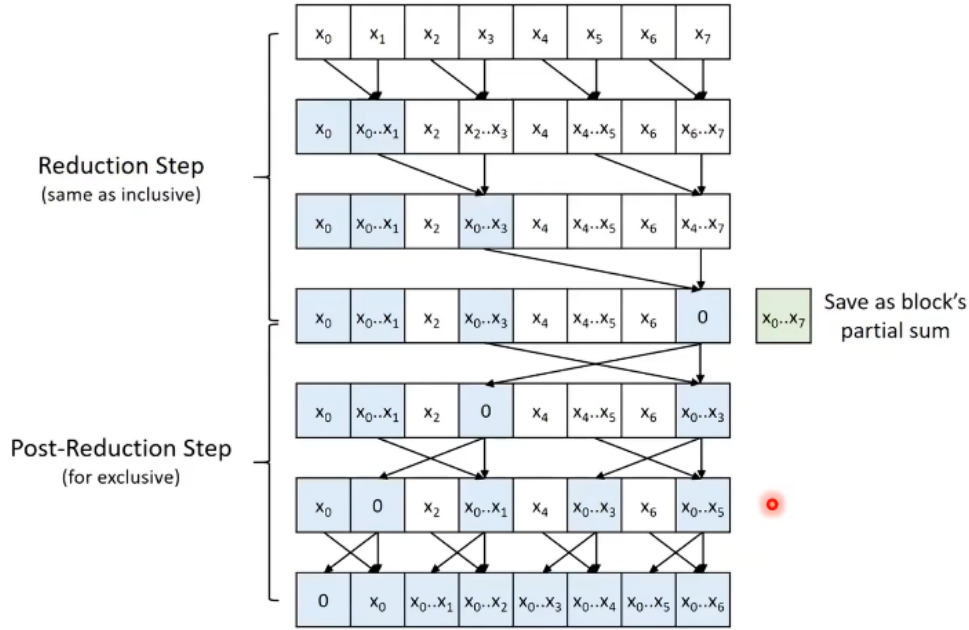


Figure 6: Brent-Kung Parallel (Exclusive) Scan

9 Histogram

Data races may result in unpredictable program output.
mutual exclusion

- CPU: locks (mutex)
- atomic operation: read-modify-write with a single ISA instruction, serialized by the hardware

atomic operations in CUDA

- atomic add: `T atomicAdd(T* address, T val)`
- function call translated to a single ISA instruction: intrinsics
- sub, min, max, inc, dec, and, or, xor, exchange, compare and swap

atomic operations on global memory have high latency

- wait for both read and write to complete
- wait if there are any other threads accessing the same location: high probability of contention
- optimize: privatization, coarsening, local counter

privatization: multiple private copies of an output are maintained, then the global copy is updated on completion

- operations on the output must be associative and commutative

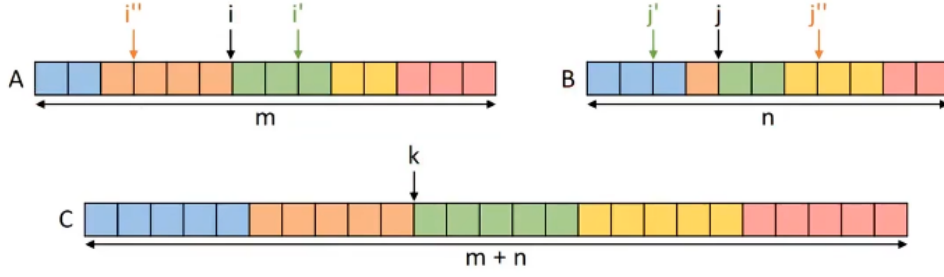


Figure 7: Finding Input Segments

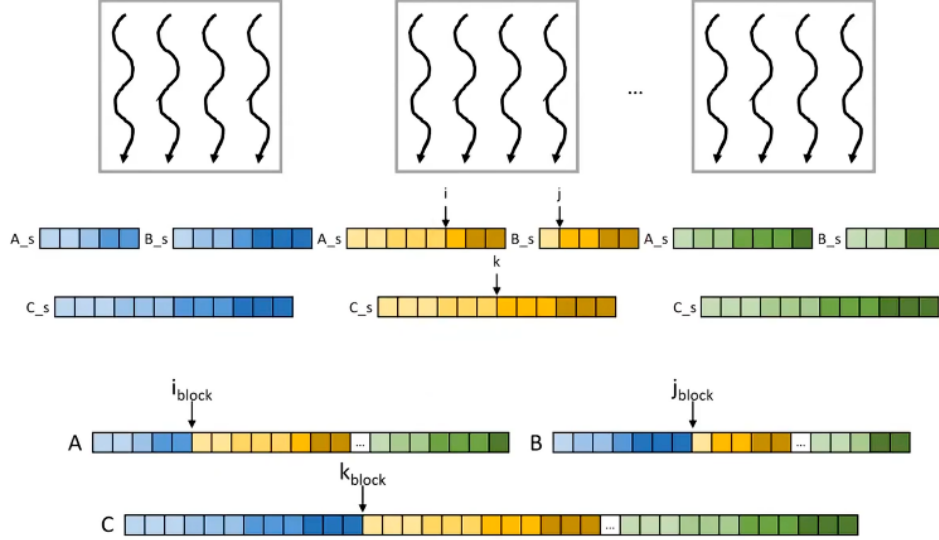


Figure 8: Shared Memory Tiling

10 Merge

ordered merge
parallelization

- each thread for a segment of output array
- finding input segments, binary search co-rank i ($j = k - i$)
 - bound on i : $\max(0, k - n) \leq i \leq \min(k, m)$
 - guess is correct: $A[i - 1] \leq B[j]$ and $B[j - 1] \leq A[i]$
 - guess is too high: $A[i' - 1] > B[j']$
 - guess is too low: $B[j'' - 1] > A[i'']$

memory accesses are not coalesced, optimization

- load the block's segment to shared memory
- do the per-thread co-rank and merge in shared memory
- already applied thread coarsening

Parallelizing Radix Sort

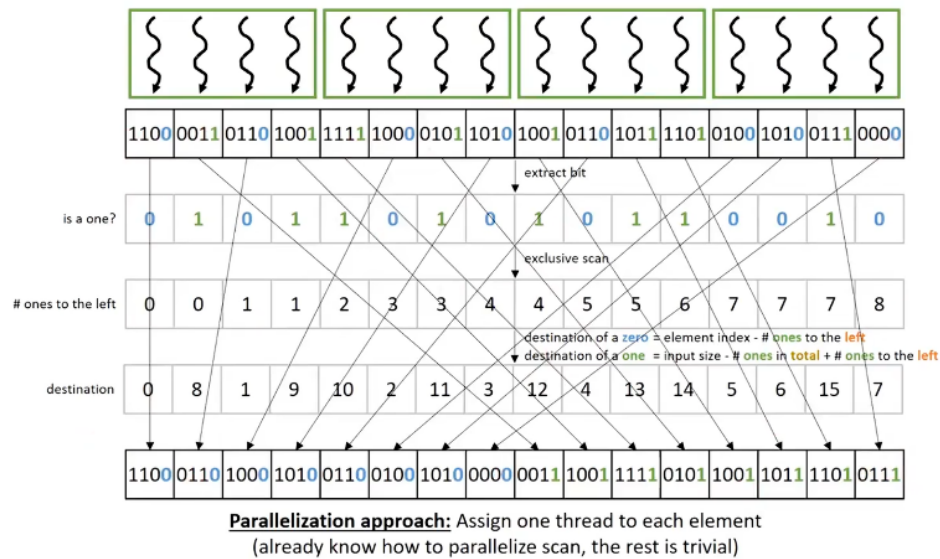


Figure 9: paralleling radix sort

11 Sort

radix sort

- distribute keys into buckets based on a radix (or base)
- distributing keys into buckets is repeated for each digit, while preserving the order from previous iterations
- using a radix that is a power of 2 simplifies processing binary numbers
 - each iteration handles a fixed slice of bits from the keys
- is not a comparison-based sort, so it's not applicable to all kinds of keys
- finding destination index
 - need to find: #ones to the left of each element
 - * destination of a zero = element index - #ones to the left
 - * destination of a one = input size - #ones in total + #ones to the left
 - * use exclusive scan
 - extract bit -> exclusive scan -> destination index -> take and store

stores are not coalesced

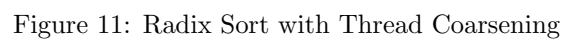
- sort locally in shared memory, then write each bucket to global memory in a coalesced manner

choice of radix value

- larger radix
 - advantage: fewer iterations
 - disadvantage: more buckets, results in poorer coalescing

thread coarsening

merge sort



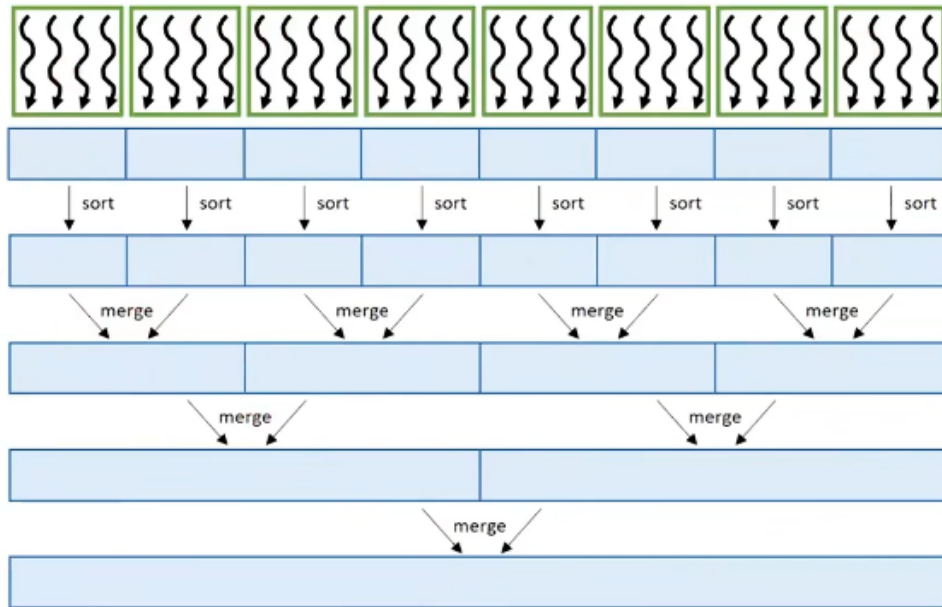


Figure 12: Merge Sort

- divide the list into sublists, sorts the sublists, then merges them (divide and conquer algorithm)
- early steps rely more on parallelism across merge operations
- later steps rely more on parallelism within merge operations

12 Sparse Matrix (COO, CSR)

sparse matrices: many elements are zero
opportunities

- do not need to allocate space for zeros (save memory)
- do not need to load zeros (save memory bandwidth)
- do not need to compute with zeros (save computation time)

sparse matrix storage formats

- Coordinate Format (COO)
- Compressed Sparse Row (CSR)
- ELLPACK Format (ELL)
- Jagged Diagonal Storage (JDS)
- ...

format design considerations

- space efficiency (memory consumed)
- flexibility (ease of adding/reordering elements)
- accessibility (ease of finding desired data)
- memory access patterns (enabling coalescing)
- load balance (minimizing control divergence)

case study: sparse matrix-vector multiplication (SpMV)

- the product of a sparse matrix and a dense vector

Coordinate Format (COO)

- stores every nonzero along with its row index and column index

SpMV/COO

- assign one thread per nonzero
- multiple threads writing to the same output, need atomic operations
- already coalesced

COO tradeoffs

- advantages
 - flexibility: easy to add new elements, nonzeros can be stored in any order
 - accessibility: given a nonzero, can easily find its row and column
 - SpMV/COO has coalesced memory access and no control divergence
- disadvantages
 - accessibility: given a row and column, hard to find all nonzeros
 - SpMV/COO requires atomic operations

Compressed Sparse Row (CSR)

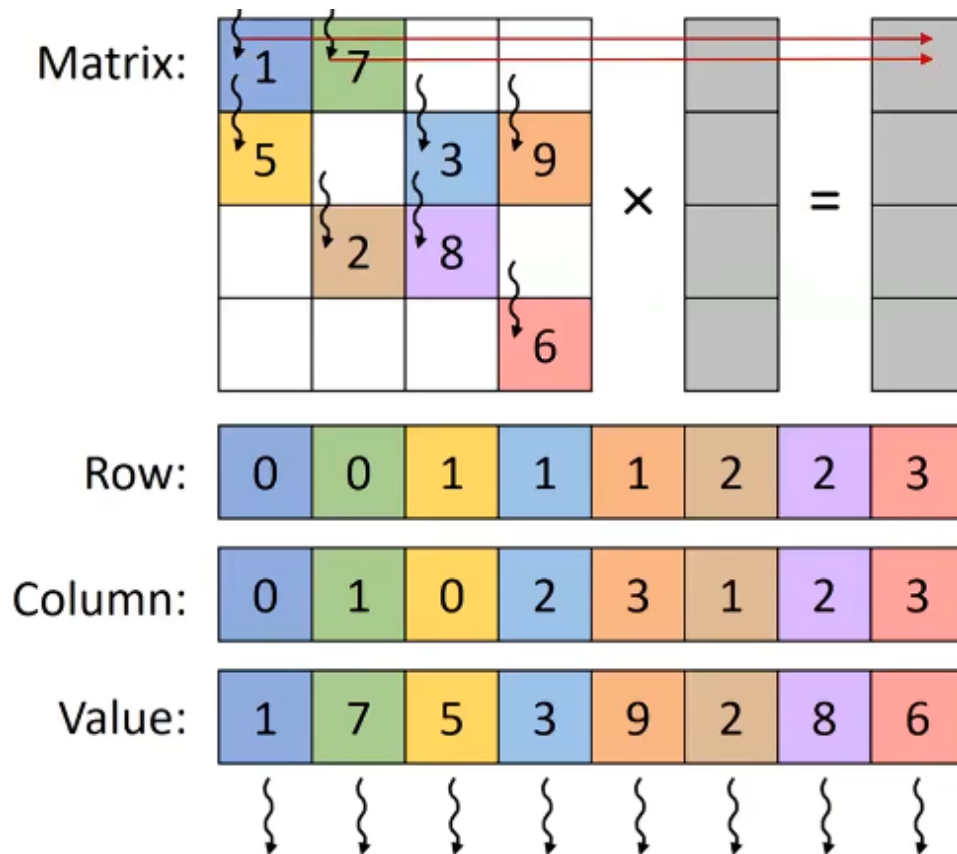


Figure 13: SpMV/COO

- stores nonzeros of the same row adjacently and an index to the first element of each row

SpMV/CSR

- assign one thread per row

CSR tradeoffs

- advantages
 - space efficiency: row pointers smaller than column indices
 - accessibility: given a row, easy to find all nonzeros
 - SpMV/CSR avoids atomics
- disadvantages
 - flexibility: hard to add new elements
 - accessibility: given nonzero, hard to find its row; given a column, hard to find all nonzeros
 - SpMV/CSR has uncoalesced memory access and control divergence

Compressed Sparse Column (CSC): like CSR but groups nonzeros by column

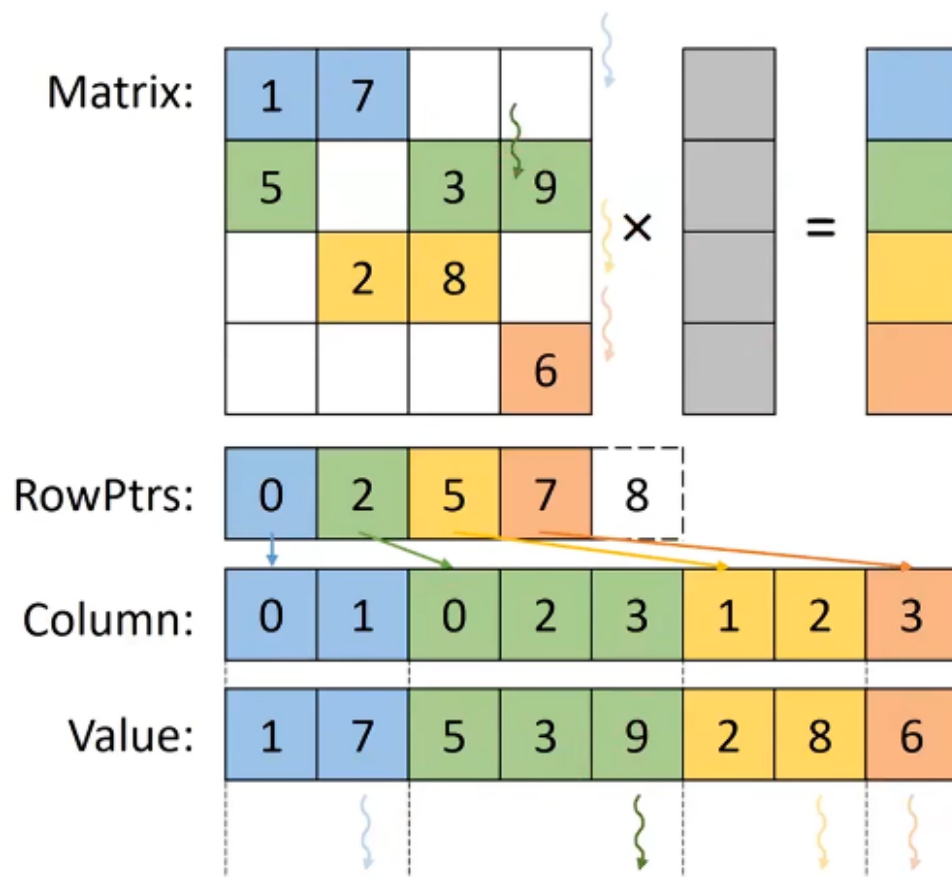


Figure 14: SpMV/CSR

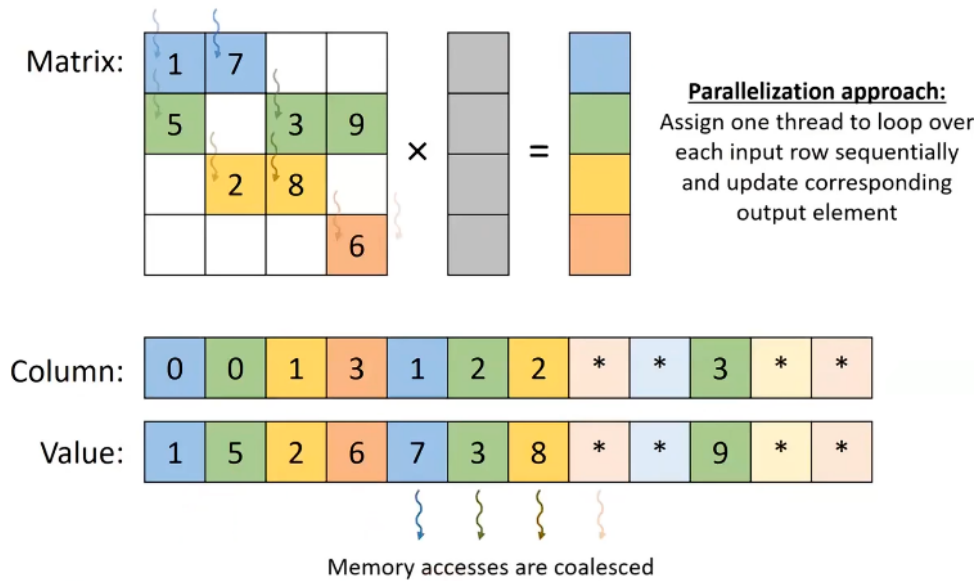


Figure 15: SpMV ELL

13 Sparse Matrix (ELL, JDS)

ELLPACK Format (ELL)

- pad each row to the same size
- store padded array in column-major order

SpMV/ELL

- assign one thread to loop over each input row sequentially

ELL tradeoffs

- advantages
 - flexibility: can add new elements as long as row not full
 - accessibility: given row, easy to find all nonzeros; given non-zero, easy to find its row and column
 - SpMV/ELL memory access is coalesced
- disadvantages
 - space efficiency: overhead due to padding
 - accessibility: given column, hard to find all nonzeros
 - SpMV/ELL has control divergence

Hybrid ELL + COO: use COO for very long rows

- benefits
 - space efficient: less padding
 - flexibility: can add new elements to any row
- still has control divergence

Jagged Diagonal Storage (JDS)

- sort rows by size and remember the original row index

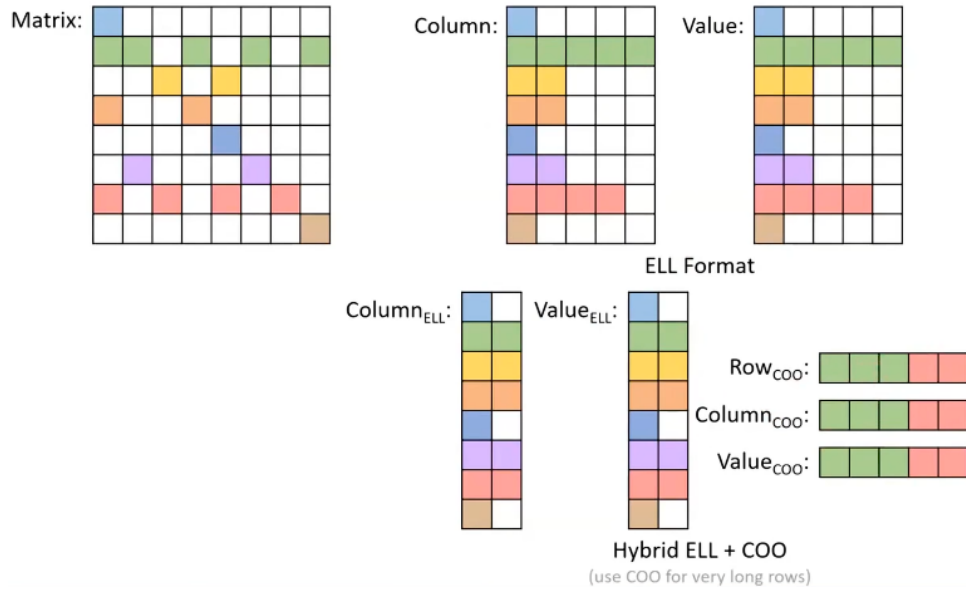


Figure 16: Hybrid ELL COO

- store nonzeros in column major
- remember where the nonzeros of each iteration start (**IterPtr**)

SpMV/JDS

- assign one thread to loop over each input row sequentially and update corresponding output element

JDS tradeoffs

- advantages
 - space efficient: no padding
 - accessibility: given a row, easy to find all nonzeros
 - SpMV/JDS memory access is coalesced
 - SpMV/JDS minimizes control divergence
- disadvantages
 - flexibility: hard to add new elements to the matrix
 - accessibility: given nonzero, hard to find row; given a column, hard to find all nonzeros

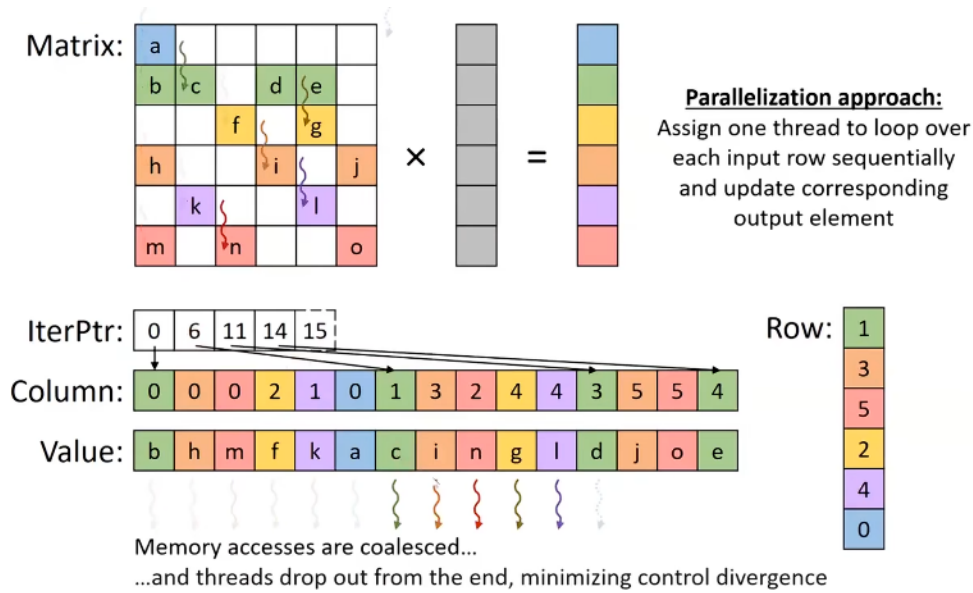


Figure 17: SpMV JDS Kernel

14 Graph Algorithms I

representing graphs: adjacency matrices
focus on

- unweighted graph: 1 for edge, 0 for no edge
- undirected graph: symmetric adjacency matrix

COO and CSR/CSC representations

- COO: array `src`, array `dst`
- CSR and CSC are equivalent because of symmetry: array `srcPtrs`, array `dst`

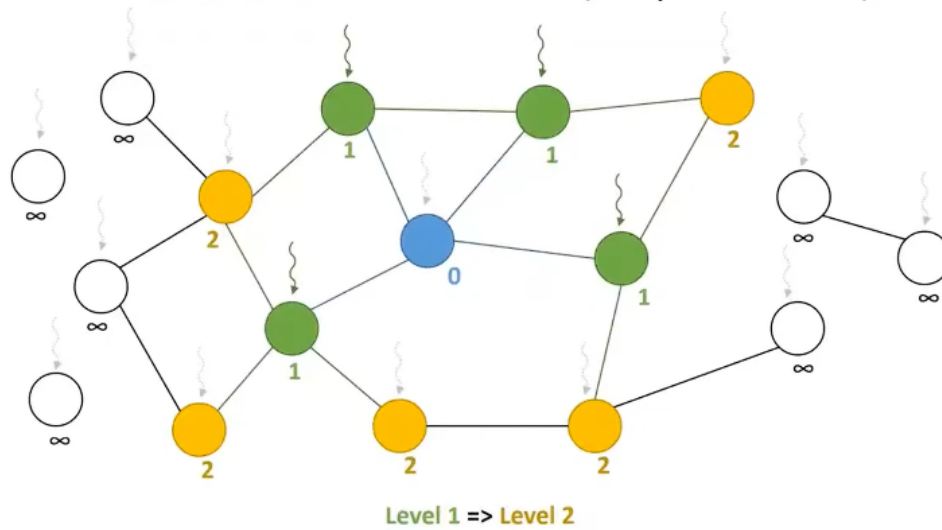
parallelizing graph processing

- vertex-centric
 - assign a thread to do something for each vertex
 - typically use CSR/CSC (might ELL, JDS, ...) : given a vertex, easy to find its neighbors
- edge-centric
 - assign a thread to do something for each edge
 - typically use COO: given an edge, easy to find its source and destination
- hybrid
 - example: given an edge, need neighbors of its source and destination, need both COO and CSR/CSC
 - * triangle counting, k-core decomposition, ...

Breadth First Search (BFS)

- vertex-centric (2 versions)
 - top-down: for every vertex in the previous level, add all its unvisited neighbors to the current level
 - * i.e. assign a thread to every parent vertex in the BFS tree

Vertex-Centric BFS (Top-Down)



Threads whose vertices are in **level 1** mark their unvisited neighbors as being in **level 2**

Figure 18: Vertex-Centric BFS (Top-Down)

- bottom-up: for every vertex, if it has not been visited, if any of its neighbors is in the previous level, add it to the current level
 - * i.e. assign a thread to every potential child vertex in the BFS tree
- direction-optimized: start with top-down then switch to bottom-up

- edge-centric

- for every edge, if its source was in the previous level, add its destination to the current level

Notice: there should be 2 threads per edge in the image because the graph is undirected, but only show one for simplicity.

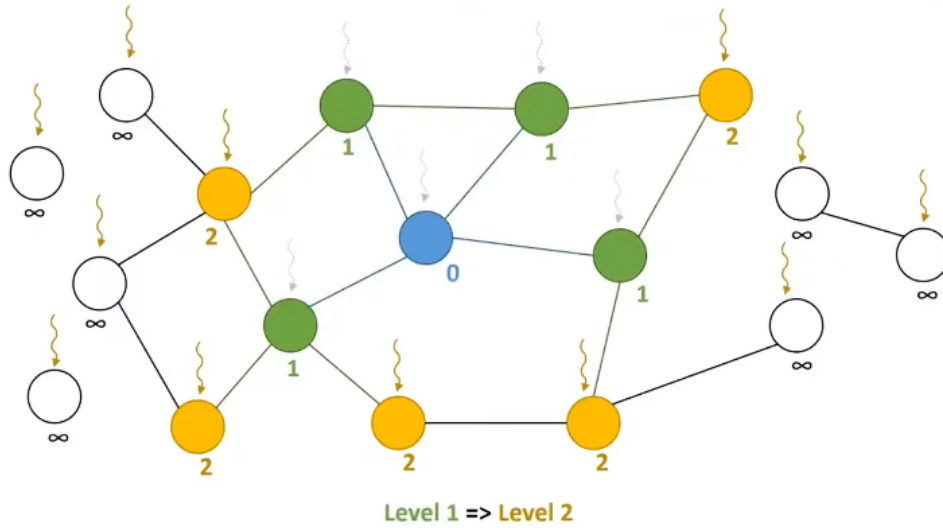
Dataset Implications

- the best parallelization approach depends on the structure of the graph
- the vertex-centric bottom-up and edge-centric approaches are better on high-degree graphs
 - e.g. social network graphs with celebrities
 - better at dealing with load imbalance
- the vertex-centric top-down approach is better on low-degree graphs
 - e.g. map of roads in a geographical area
 - can be improved by launching more threads to process neighbors of high-degree vertices

Linear Algebraic Formulation of Graph Problems

- with a few tweaks, BFS can be formulated as SpMV
 - use visited list as input/output vector
 - a few other operations to get wanted
- Many graph problems can be formulated in terms of sparse linear algebra computations
 - advantage: leverage mature and well-optimized parallel libraries for high performance sparse linear algebra

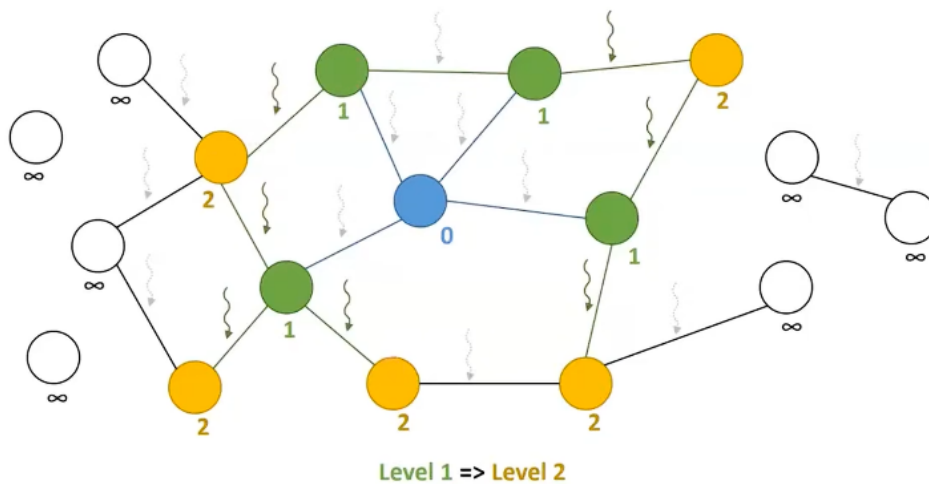
Vertex-Centric BFS (Bottom-Up)



Threads whose vertices are unvisited mark their vertices as being in **level 2** if any of their neighbors are in **level 1**

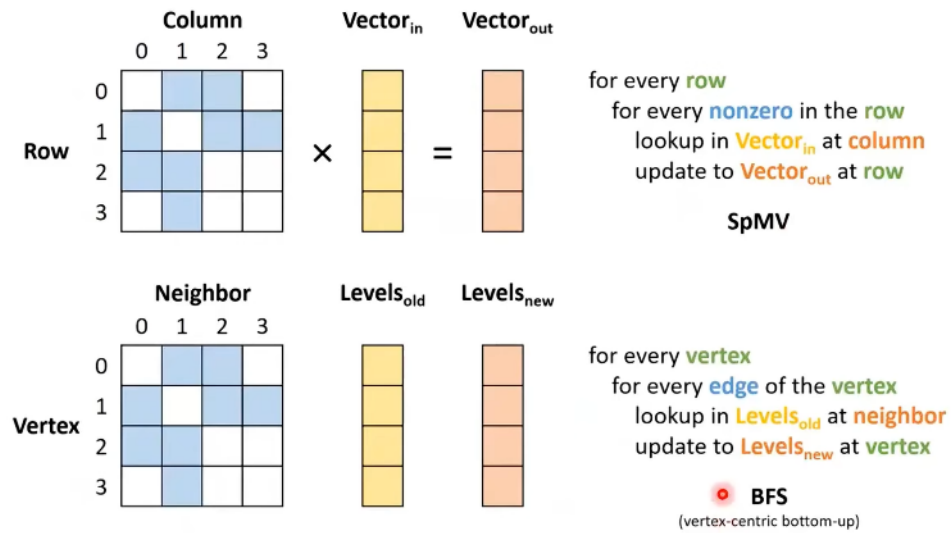
Figure 19: Vertex-Centric BFS (Bottom-Up)

Edge-Centric BFS



Threads whose edge's source vertex is in **level 1** and whose edge's destination vertex is unvisited mark their edge's destination vertex as being in **level 2**

Figure 20: Edge-Centric BFS



Vertex-centric top-down and edge-centric also correspond to other ways of performing SpMV

Figure 21: Similarity between BFS and SpMV

- disadvantage: not always the most efficient way to solve the problem

Redundant Work

- approaches so far check every vertex/edge on every iteration for relevance
- easy to implement, highly parallel, no synchronization across threads
- weakness: a lot of unnecessary threads/work
 - many threads will find that their vertex/edge is not relevant for this iteration and just exit

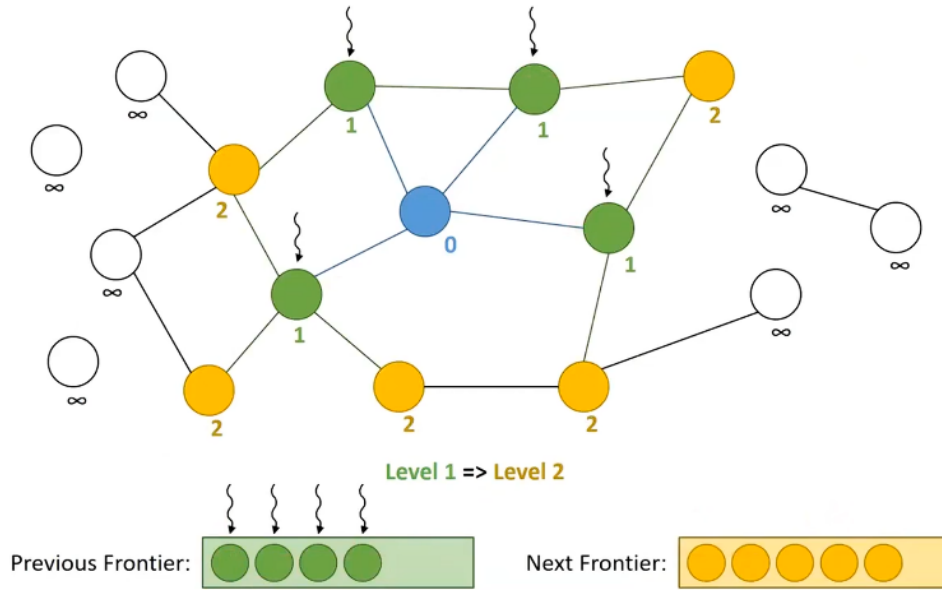


Figure 22: Vertex-Centric Frontier-Based BFS

15 Graph Algorithms II

Remark: if not symmetric, top-down BFS need CSR representation, while bottom-up BFS need CSC representation, (treating rows as source) dir-opt BFS need both CSR and CSC.

Reduce Redundancy

- objective: only check the vertices that are part of the previous level
- approach: each level places the vertices it visits in a queue for the next level to process
 - vertices enqueued at each level form that level's frontier
- overhead: synchronization across threads to add to a shared queue

Queue Privatization

- all threads atomically increment the same global counter to insert elements into the queue, high latency due to global memory access and serialization due to high contention.
- optimization: each thread block maintains a private queue (in shared memory, with the counter) and commits entries to the global queue on completion

Minimizing Launch Overhead

- need to synchronize between levels
- so far, we have launched a new grid for each level
 - overhead of launching new grid and copying counter
- optimization: if consecutive levels have few enough vertices that can be executed by one thread block:
 - execute multiple levels in one single-block grid and synchronize between levels using `__syncthreads()`
 - reduce total number of grid launches

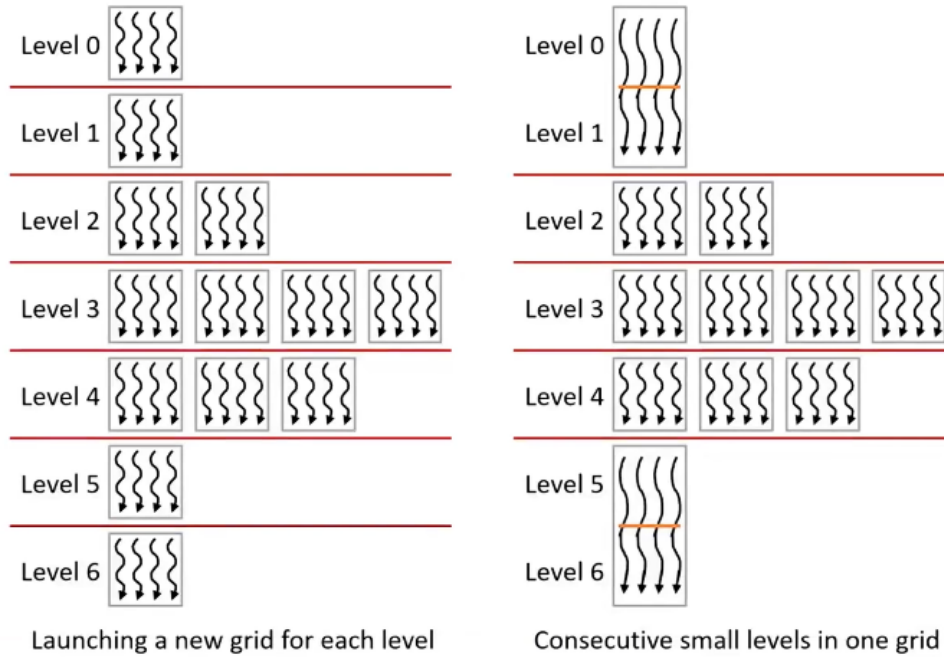


Figure 23: Minimizing Launch Overhead

16 Warp Synchronization

16.1 intra-warp synchronization

take advantage of the special relationship between threads in the same warp to synchronize quickly
 CUDA has several built-in functions to synchronize between warps

- sharing (shuffling) data across threads
- voting across threads

16.2 shuffle

built-in warp shuffle functions enable threads to share data with other threads in the same warp

- faster than shared memory and `__syncthreads()` cross the block
- variants
 - `__shfl_sync()`: direct copy from indexed lane
 - `__shfl_up_sync()`: copy from a lane with lower ID relative to the caller
 - `__shfl_down_sync()`: copy from a lane with higher ID relative to the caller
 - `__shfl_xor_sync()`: copy from a lane based on a bitwise XOR of own lane ID

example: reduction/scan

- When one warp remains, use `__shfl_down_sync()` to share data from registers

16.3 voting

built-in warp vote functions enable threads to vote on which threads in the warp satisfy some condition

- `__all_sync(unsigned mask, predicate)`: check if predicate is nonzero for all threads
- `__any_sync(unsigned mask, predicate)`: check if predicate is nonzero for any thread

- `__ballot_sync(unsigned mask, predicate)`: return mask of which threads evaluate predicate to nonzero
- `__activemask()`: return mask of which threads are active

example: enqueue, have only one thread in the warp increment the counter on behalf of the others

1. assign a leader thread
 - find the active threads in the warp using `__activemask()`
 - designate the first active thread as the leader using `__ffs(mask) - 1`
 - index of `__ffs(mask)` starts from 1, returns 0 if no bits are set
2. find how many threads need to enqueue
 - use `__popc(mask)` to count the number of set bits in the mask
3. have the leader perform the atomic operation
 - threads in the same warp are consecutive, so `threadIdx.x % WARP_SIZE` gives the position in the warp
4. broadcast the result to the other threads
 - use `__shfl_sync(mask, j, leader)`
5. find offset of each active thread and store the result
 - use bitwise AND to find the number of previous active threads

Gemini's suggestions:

1. Replaced `__activemask()` with `__ballot_sync(0xffffffff, is_active)`: On modern GPU architectures (Volta and above), independent thread scheduling can cause `__activemask()` to return only a subset of active threads taking a divergent branch. Using `__ballot_sync` guarantees all evaluating threads in the warp are grouped together, preventing multiple partial `atomicAdds`.
2. Changed `1 << ...` to `1u << ...`: This avoids undefined behavior from signed integer overflow when the thread index is 31.

17 Pinned Memory and Streams

17.1 Pinned Memory

Direct Memory Access (DMA) allows some hardware to access memory without involving the CPU

- copying between CPU and GPU uses DMA
- advantage: CPU can be utilized for other tasks while the memory copy is taking place
- DMA use physical addresses to access memory, cannot detect if the OS swaps a virtual page with another virtual page at the same physical address
- to avoid data corruption, the OS must not be allowed to swap pages being accessed by DMA
- pages to be accessed by DMA must be allocated as pinned memory (also called page-locked memory), marked as non-swappable by the OS

cudaMemcpy

- host to device: CPU copies data to a pinned memory buffer, DMA copies data from pinned memory buffer to the device
- device to host: DMA copies data from the device to a pinned memory buffer, CPU copies data from pinned memory buffer
- disadvantage: every cudaMemcpy is actually two copies, which incurs overhead

faster copies: allocate host arrays directly in pinned memory

- `cudaError_t cudaMallocHost(void **devPtr, size_t size)`
- `cudaError_t cudaFreeHost(void *devPtr)`
- use with caution: only use for frequently copied data if overhead is high
- allocating too much pinned memory degrades overall system performance by reducing the virtual pages available for swapping

17.2 Stream

Typical system is simultaneously capable of

- executing grids on the GPU
- copying from host to device
- copying from device to host

Example: previous timeline of vector addition

- copy A to device
- copy B to device
 - not utilizing GPU and `copyDeviceToHost`
- execute grid to compute $A+B$
 - not utilizing copies
- copy C from device
 - not utilizing GPU and `copyHostToDevice`

pipelining vector addition

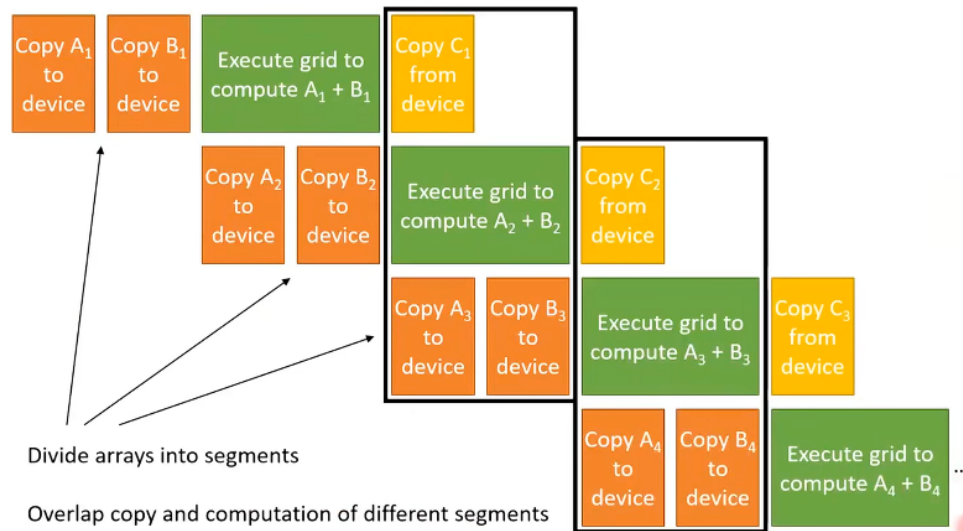


Figure 24: Pipelining Vector Addition

- divide arrays into segments
 - overlap copy and computation of different segments
- streams and asynchronous copies
- parallelism between grids and memory copies is achieved by using **streams**
 - tasks in different streams are executed in parallel
 - tasks in the same stream are serialized
 - if no stream is specified, tasks go into the default stream
 - to overlap memory copies with host execution, use **asynchronous memory copies**
 - `cudaError_t cudaStreamCreate(cudaStream_t *pStream)`
 - `cudaError_t cudaMemcpyAsync(void *dst, const void *src, size_t count, cudaMemcpyKind kind, cudaStream_t stream=0)`
 - `kernel<<<blocks, threads, smem, stream>>>(...)`
 - `cudaEvent_t` can be used to measure the time taken in different streams

18 Dynamic Parallelism

Dynamic Parallelism: CUDA dynamic parallelism refers to the ability of threads executing on the GPU to launch new grids

Nested Parallelism: Dynamic parallelism is useful for programming applications with nested parallelism, where each thread discovers more work that can be parallelized; particularly useful when the amount of nested work is unknown, so enough threads cannot be launched up front.

applications

- nested parallel work is **irregular** (varies cross threads)
 - e.g. graph algorithms (each vertex has a different number of neighbors)
 - e.g. Bezier curve (each curve needs different number of points to draw)
- nested parallel work is **recursive** with unknown depth
 - e.g. tree traversal algorithms (e.g., quadtrees and octrees)
 - e.g. divide and conquer algorithms (e.g., quicksort)
- the device code for calling a kernel to launch a grid is the same as the host code
- memory is needed for buffering grid launches that have not started executing
 - The limit on the number of dynamic launches is referred to as the **pending launch count**
 - By default, the runtime supports 2048 launches, and exceeding this limit will cause an error
 - The limit can be increased by setting `cudaDeviceSetLimit(cudaLimitDevRuntimePendingLaunchCount, <new limit>)`

example: graph frontier BFS

- Directly launching child kernel behaves worse
1. for device launches, threads in the same block share the same default stream (serialized)
 - improve by creating a different stream per thread
 - use `cudaStreamCreate` as host, or
 - use compiler flag `--default-stream per-thread`
 1. Pitfalls
 - launching very small grids may not be worth the overhead (more efficient to serialize)
 - launching too many grids causes queuing delays
 2. Optimization: apply a threshold to the launch
 - only launch the large grids that are worth the overhead, and serialize the rest
 - threshold value is data dependent and can be tuned
 3. Optimization: aggregate launched
 - have one thread collect the work of multiple threads and launch a single grid on their behalf

Offloading Driver Code

- free up the host to do other things
- notice: for single thread, CPU code is faster than GPU code, so it's unworthy in many cases

Memory Visibility

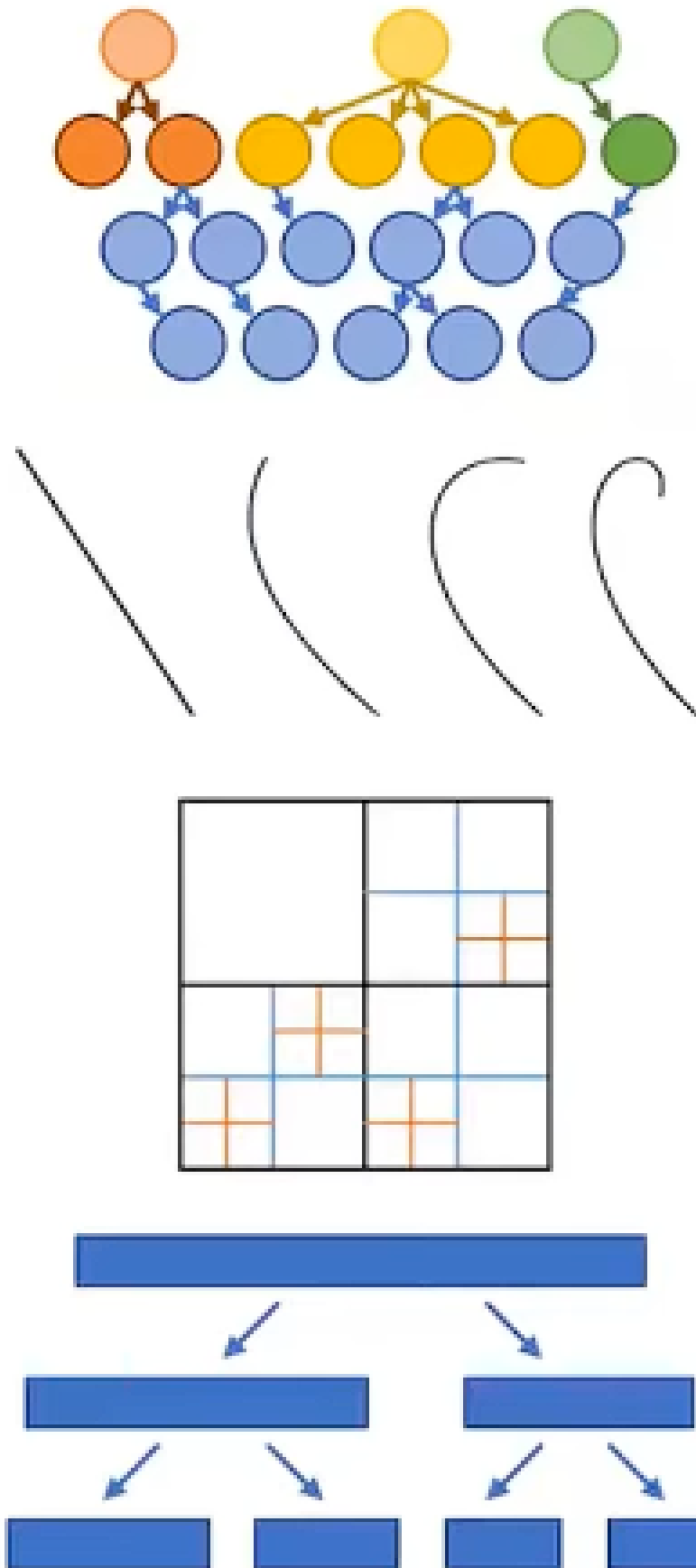


Figure 25: Applications of Dynamic Parallism

- Operations on global memory made by a parent thread before the launch are visible to the child threads
- Operations made by the child are visible to the parent after the child returns and the parent has synchronized
- A thread's local memory and a block's shared memory cannot be accessed by child threads

Nesting Depth: how deeply dynamically launched grids may launch other grids

- determined by the hardware (typically 24)

19 CUDA Potpourri

19.1 Multi-GPU programming

multiple GPUs on the same node

- `cudaGetDeviceCount`
- `cudaSetDevice` to select one
- typically use multiple CPU threads with one driving each GPU (e.g. using OpenMP)

multiple GPUs on multiple different nodes

- typically use MPI for programming multiple nodes, and each MPI rank interacts with its GPU normally
- important consideration: overlapping MPI communication with kernel execution

19.2 Interconnect

- PCIe: general, widely supported
- NVLink: faster between multi-GPUs, but only supported on some GPUs and some CPUs

19.3 Memory management

Unified Virtual Addressing (UVA)

- CPU $\longleftrightarrow$

Main Memory $\longleftrightarrow$ Global Memory

 $\longleftrightarrow$ GPU

UVA space
- advantage: data location is known from the pointer value, so do not need to specify copy direction
 - `cudaMemcpyDefault`
- simpler code
- performance depends on situation

Zero-copy memory: enables devices to directly access host memory

- memory has to be pinned
- `cudaHostAlloc(&ptr, size, cudaHostAllocMapped)` to allocate the memory
- `cudaHostGetDevicePointer` to get corresponding device pointer
 - unnecessary if using UVA

19.4 Events

- synchronizing after every copy or kernel call interferes with execution and slows things down
- Events can be used to collect timing information without synchronizing
 - `cudaEventCreate(&event)`
 - `cudaEventRecord(event, stream=0)`
 - `cudaEventSynchronize(event)`
 - `cudaEventElapsedTime(&result, startEvent, endEvent)`
 - `cudaEventDestroy(event)`
- profilers can also be used

19.5 Tensor cores

programmable matrix multiplication and accumulation units

- accelerate DNN workloads
- in Volta v100, 640 cores (8 per SM), each core is capable of a 4x4 matrix multiply-accumulate operation $D = A * B + C$

19.6 Libraries

examples of libraries for common parallel primitives

- Thrust: reduction, scan, filter, and sort
- cuBLAS: basic dense linear algebra
 - NVBLAS: multi-GPU library on top of cuBLAS
- cuSPARSE: sparse-dense linear algebra
- cuSOLVER: factorization and solve routines, on top of cuBLAS and cuSPARSE
- cuDNN: deep neural networks
 - used by Caffe, TensorFlow, PyTorch, MxNet, etc.
- nvGRAPH: graph processing
- cuFFT: Fast Fourier Transform
- NPP: signal, image, and video processing

19.7 Other programming interfaces

- OpenCL: similar to CUDA, but open source and more portable across non-NVIDIA GPUs
- OpenACC: directive-based GPU programming
 - annotate code similar to OpenMP
- C++AMP: implementing GPU programs directly in C++

19.8 Other hardware

other GPUs

- AMD (Radeon)
- ARM (Mali)
- Intel

integrated GPUs

FPGA

19.9 Comparison with CPU

more fair to compare with parallel and vectorized CPU code

20 Matrix Multiplication Optimization

the simple matrix multiplication kernel is memory bound

- arithmetic intensity is 0.25 FLOP/B
- higher in practice due to caches, but still memory bound

but matrix multiplication has the potential to be highly compute bound (so it suits GPU well)

- for N - N square matrices, data accessed is $12N^2$ B, operations is $2N^3$ FLOP, so arithmetic intensity is $N/6$ FLOP/B
- h100 approximately 20 FLOP/B

review: tiled matrix-matrix multiplication

- input in shared memory, output in registers
- for n - n square tiles (ignore storage), $(2n^3 \text{ FLOP}) / (8n^2 \text{ B}) = 0.25n \text{ FLOP/B}$, 8 FLOP/B for $n = 32$
- increasing n

Let the tiles of A, B, C have size $m \cdot k$, $k \cdot n$, $m \cdot n$

- arithmetic intensity: $(mn)(k \text{ adds} + k \text{ muls}) / ((mk + kn \text{ values})(4 \text{ B})) = 0.5mn/(m + n) \text{ FLOP/B}$
- increasing m and n increases reuse ratio, keeping k small prevents increasing demand on shared memory, but stress on registers
- typically use $m = n = 128$, $k = 8$, achieving 32 FLOP/B, k not too small to utilize caches
 - use 256 threads per block, 8x8 elements per thread

By default, local memory arrays are placed in the global memory

- with loop unrolling, variable indices are converted to constant indices which allows the compiler to promote the arrays to registers
- e.g. `__forceinline__` with `#pragma unroll`

additional optimizations

- vector loads/stores: improve global memory bandwidth utilization
 - each thread issues 1 vector load which loads consecutive 16B (4 elements)
 - advantage: fewer loads/stores instructions, better store coalescing
- register tiling: reduce shared memory access latency
 - iterating over input tiles one strip at a time and load the strip into registers
- coalescing stores: improve global memory bandwidth utilization
 - rearrange tiles: divide to 4 quadrants
- eliminating bank conflicts: reduce shared memory access latency
 - avoid multiple threads accessing the same bank of shared memory at the same time
 - add padding (A tile), already avoided by rearranging (B tile)
- software pipelining: improve latency tolerance
 - advanced matrix multiplication kernels usually have low occupancy due to high register usage (register tiling of output tile and input strips, aggressive loop unrolling and instruction scheduling)

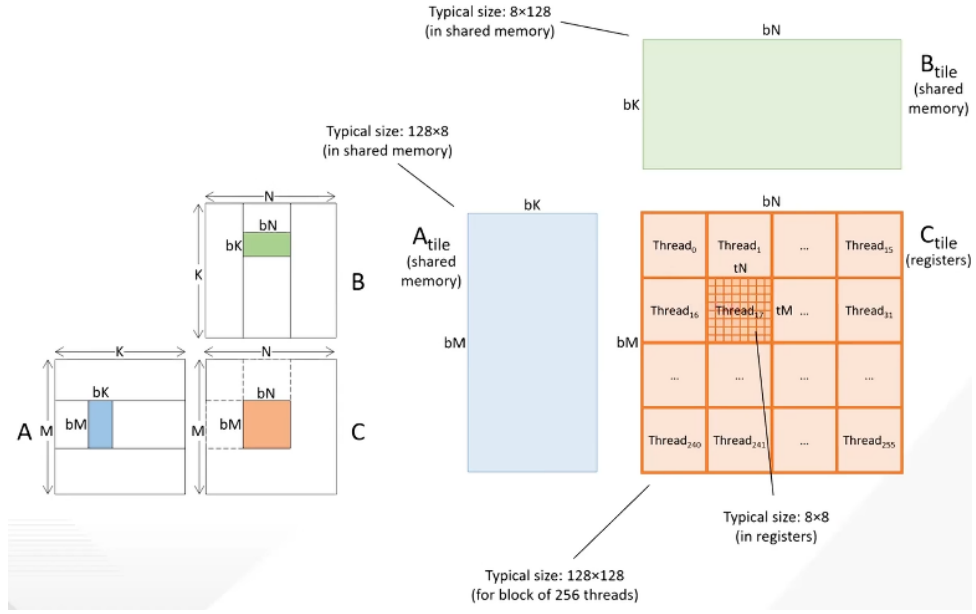


Figure 26: Use Large Tiles

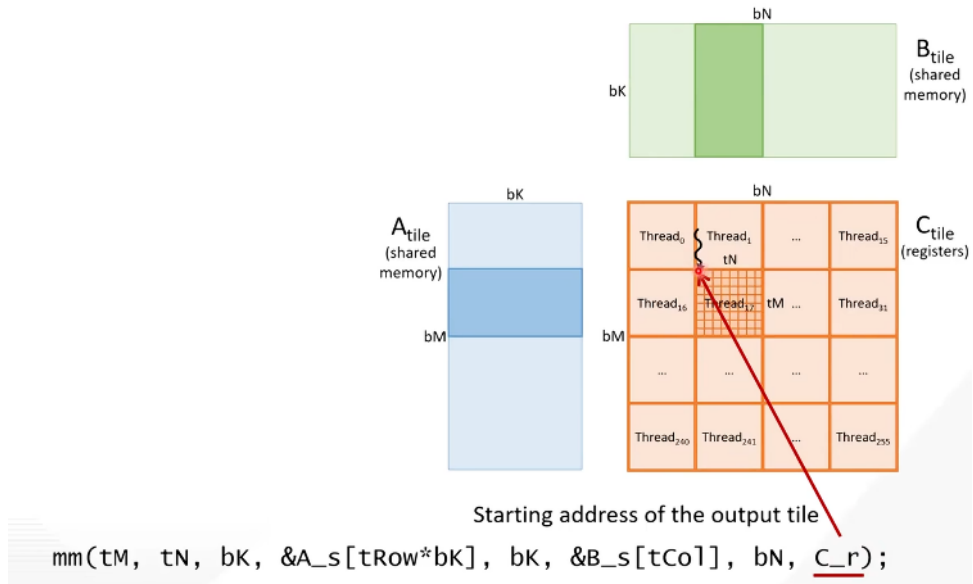


Figure 27: Computing Thread Tile

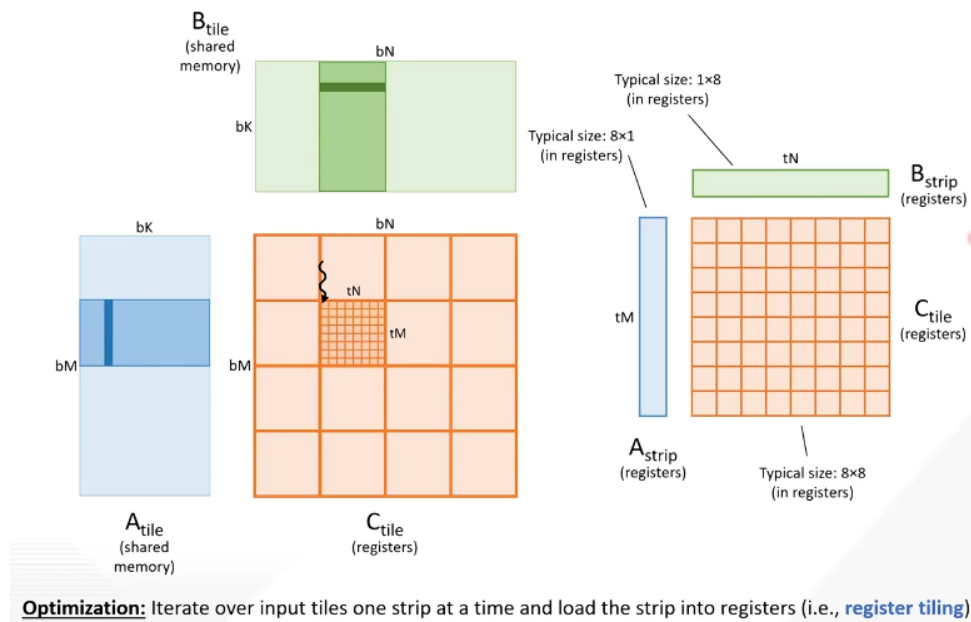


Figure 28: Register Tiling

- each thread may use up to the maximum allowed 255 registers/thread, with 64k registers/SM, that limits occupancy to 256 threads/SM, 12.5% of the maximum 2048 threads/SM
- drawback of low occupancy: fewer warps to hide core and memory latency
- alternative optimizations for hiding latency to compensate for low occupancy
 - * loop unrolling already helps hide core latency
 - * vector loads/stores help hide memory latency
 - * software pipelining and warp specialization
- all iterations have memory-bound phases and compute-bound phases, but there are no other blocks on the SM to tolerate the phase behavior
- true and false dependencies by synchronization
- software pipelining: rearrange loop so that compiler can overlap a tile with loading the next tile
- apply warp specialization: allows hardware to interleave the compute and memory instructions from different warps dynamically
- using specialized hardware units
 - tensor memory accelerator (TMA): asynchronous copy engine between global and shared memory
 - * avoids busying cores with instructions for copying data
 - * avoids using registers for copying data from global to shared memory
 - tensor cores: perform small matrix multiplications
 - * typically use in cuBLAS and CUTLASS

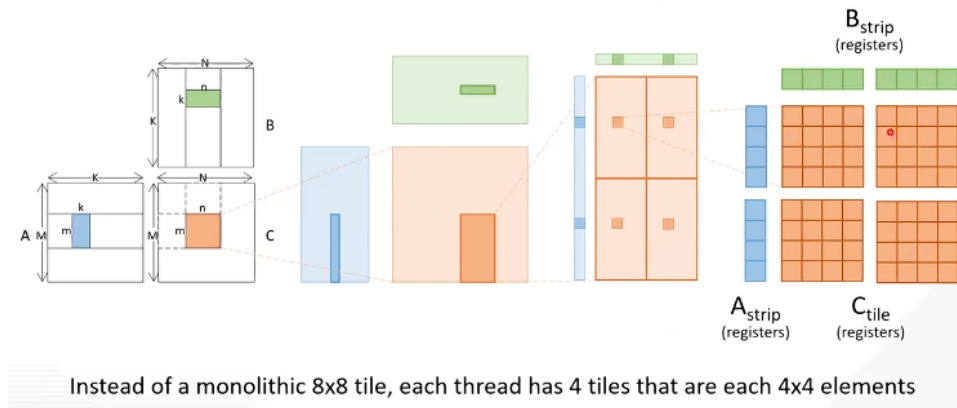


Figure 29: Rearranging Tiles to Coalesce Stores

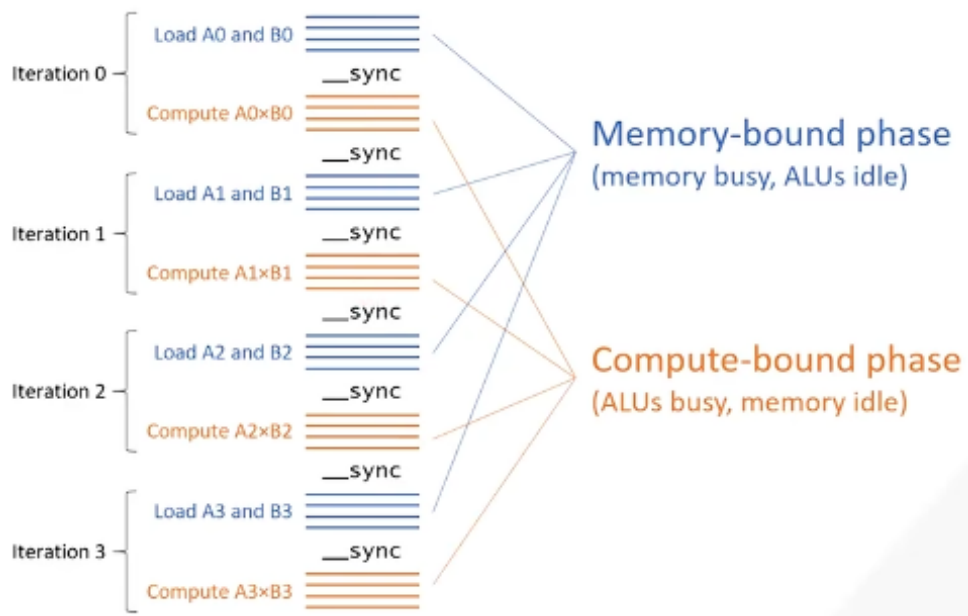


Figure 30: Motivation for Software Pipelining